

Temporal Databases

Group Name: Query Stack

Ashok Chandra R	23CS10059
Monica Reddy N	23CS10046
Rachamsetty Harsha Vardhan	23CS10056
Miryala Avinash Kumar	23CS10045

1. Introduction

Financial markets produce large amounts of time-series data continuously. Storing and querying this data efficiently is not straightforward, especially when the ingestion rate is high and the dataset grows over time. Standard database setups tend to slow down under these conditions without proper tuning.

This project looks at how a relational database like PostgreSQL can be set up to handle financial tick data well. We use techniques like batch ingestion, partitioning, indexing, and compression to improve both write and read performance. The goal is to build a working prototype and measure how much each optimization actually helps.

2. Abstract

Modern financial markets produce huge amounts of high-frequency time-series data in the form of trade ticks. Handling this data efficiently is difficult because it arrives very quickly and requires large storage capacity. This project focuses on building a lightweight time-series database prototype designed specifically for financial tick data.

The system imitates real market activity by generating synthetic tick data for multiple stock symbols at a configurable rate. Each tick records a timestamp, stock symbol, price, and trading volume. To handle the continuous stream of incoming data, the system supports high-speed ingestion and organizes the data through a hierarchical aggregation process. Raw tick data is periodically converted into Open-High-Low-Close-Volume (OHLCV) bars at different time intervals such as one second, one minute, and one hour. This layered aggregation structure allows analytical queries to run more efficiently without scanning large amounts of raw data.

To manage storage effectively, the system applies techniques such as data partitioning, batch insertion, and compression of older data. These methods help scale storage while keeping query performance fast for both recent and historical data. The prototype shows how time-series financial data can be modeled, stored, and queried efficiently within a relational database.

Overall, the project demonstrates key concepts used in real financial data platforms, including continuous data ingestion, time-based aggregation, and storage optimization for large-scale time-series workloads.

3. Tech Stack

3.1. Backend

- Python
- NumPy

3.2. Database

- PostgreSQL
- Pgbench for benchmarking

3.3. Frontend

- StreamLit – Python Library

4. Key Features

- **DBMS Performance Dashboard:** A dedicated UI view comparing the execution latency of standard PostgreSQL queries versus our hypertables to mathematically prove the platform’s efficiency.
- **Continuous Aggregates in OHLCV form:** Implementation of aggregates in order to strike a balance between short term precision and long term retention in (open, high, low, close, volume) data format.
- **Time Acceleration:** To simulate real time data insertion of consecutive days, we ensure that the virtual clock runs faster than the “time of day” clock.
- **Interactive Data Visualization:** The frontend provides charts to visualize price and volume trends of different stock symbols using the stored time-series data.
- **Symbol and Time Range Selection:** Users can select stock symbols and specify time ranges to query and analyze historical tick or aggregated data.

5. Optimizations

- **SQL Triggers:** Automated scripts to create child tables when the virtual clock hits a new day instead of manually writing `CREATE TABLE ...`
- **Declarative Partitioning:** Child tables created based on the partitions by date to facilitate bulk deletion of rows corresponding to a particular date instead of iterating through the entire table.
- **Batch ingestion:** We minimize the overhead of transaction commits by grouping thousands of events into a single bulk operation.
- **In-Memory buffer:** An “In-Memory Buffer” (like a NumPy Array) and a bulk insertion method, will be faster.
- **Compression:** The older (more than 7 days) raw ticks are compressed to reduce their memory footprint (reduced by 5x compared to directly storing raw data).

- **Indexing:** Indexing to ensure because our query patterns are almost always range-based filters on time. Without defining these indexes, the $O(N)$ cost of full table scans would make real-time aggregation impossible as the dataset grows.

6. Types of Queries (common)

1. Time Range Query : Retrieves tick or aggregated data between two timestamps. This is useful for analyzing market activity during a specific time interval.
2. OHLC aggregation: Generates candlestick bars (Open, High, Low, Close, Volume) from raw tick data. These bars are commonly used for financial charting and market analysis.
3. Moving average: Computes the average price over a fixed time window using aggregated data. It helps smooth short-term fluctuations and identify overall price trends.
4. Volume analysis: Calculates the total trading volume of a stock within a given time period. This helps identify periods of high market activity or unusual trading behavior.
5. Latest price: Fetches the most recent tick for a stock symbol. This is used to determine the current market price of a stock.
6. Price volatility: Measures how much the stock price fluctuates over a time interval. It is useful for understanding risk and identifying highly volatile stocks.

7. Weekly Schedule

Week 1 — Build the basic system (MVP)

Goal: Make a simple working system

- Write a Python program to generate tick data (10 stock symbols, ~100 ticks/sec).
- Create the basic database table and schema and also store generated ticks in the database.
- Test simple queries to check if the data is being stored correctly.

Week 2 — Add basic aggregation

Goal: Start processing the data

- Create aggregation tables.
- Convert raw tick data into OHLCV bars.
- Implement simple aggregation pipeline.

Week 3 — Improve performance

Goal: Try various methods to increase throughput

- Implement batch insertion instead of inserting one row at a time.
- Add indexes on (symbol, timestamp) for faster queries.
- Develop and refine front-end features.

Week 4 — Storage optimization

Goal: Handle large amounts of data

- Implement partitioning and time-based storage for ticks.
- Implement archiving of old ticks.

Week 5 — Testing and report

Goal: Evaluate and document the system

- Run benchmark tests (ingestion speed, query speed).
- Perform stress testing with large tick datasets.
- Compare performance before and after optimizations.
- Final report and documentation.

8. Outcomes

By the end of this project, we expect to have a working prototype that can ingest financial tick data at high frequency and store it efficiently in PostgreSQL. The multi-resolution OHLCV aggregation pipeline should allow faster analytical queries without scanning raw tick data every time. We expect the combination of batch ingestion, indexing, and partitioning to show measurable improvements in both write and read performance. Compression of older ticks should bring down storage usage significantly. Overall, this project should give us a clearer picture of how far a standard relational database can be pushed for time-series workloads with the right set of optimizations.